

PREDICTIVE ANALYTICS
PROJECT REPORT
(Project Semester Oct'25 -Jan'26)

Heart Failure Mortality Prediction



Submitted by

Saurabh Kumar

Registration No: 12314234

Programme and Section: P132, K23KY

Course Code: INT234

Under the Guidance of

Dr. Vikas Mangotra (UID 31488)

Discipline of CSE/IT

Lovely School of Computer Science and Engineering

Lovely Professional University, Phagwara

CERTIFICATE

This is to certify that **Saurabh Kumar** bearing Registration no. **12314234** has completed **INT234** project titled, “**Heart Failure Mortality Prediction**” under my guidance and supervision. To the best of my knowledge, the present work is the result of his/her original development, effort and study.

Signature and Name of the Supervisor

Designation of the Supervisor

School of Computer Science and Engineering

Lovely Professional University

Phagwara, Punjab.

Date: 15/12/25

DECLARATION

I, Saurabh Kumar, student of B.Tech under CSE/IT Discipline at, Lovely Professional University, Punjab, hereby declare that all the information furnished in this project report is based on my own intensive work and is genuine.

Date: 15/12/25

Signature

Registration No. 12314234

Saurabh Kumar

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to Dr. Vikas Mangotra, my project supervisor, for his valuable guidance, continuous encouragement, and constructive feedback throughout the development of this project. His insights and support played a crucial role in shaping the analytical direction, methodology, and overall quality of the work.

I am also thankful to the faculty members of the School of Computer Science, Lovely Professional University, for providing a supportive academic environment and the necessary resources required for the successful completion of this project.

Lastly, I acknowledge the use of the publicly available Heart Failure Clinical Records dataset and open-source machine learning libraries that enabled effective data preprocessing, model development, evaluation, and visualization for mortality prediction. This project has been an enriching learning experience, and I am grateful to everyone who contributed directly or indirectly to its successful completion.

TABLE OF CONTENTS

1. Introduction

- 1.1 Background of Predictive Analytics
- 1.2 Importance of Heart Failure Mortality Prediction
- 1.3 Motivation Behind the Project
- 1.4 Problem Statement
- 1.5 Objectives of the Study
- 1.6 Scope of the Project

2. Source of Dataset

- 2.1 Overview of the Dataset
- 2.2 Dataset Origin and Relevance
- 2.3 Description of Data Attributes
- 2.4 Data Size and Structure
- 2.5 Assumptions and Limitations of the Dataset

3. Dataset Preprocessing

- 3.1 Need for Data Preprocessing
- 3.2 Handling Missing Values
- 3.3 Data Cleaning and Validation
- 3.4 Feature Scaling and Normalization
- 3.5 Final Dataset Preparation

4. Analysis on Dataset

Objective 1: Exploratory Data Analysis of Heart Failure Data

- 4.1.1 General Description
- 4.1.2 Analysis Results
- 4.1.3 Visualization

Objective 2: Machine Learning Model Development

- 4.2.1 Models Implemented
- 4.2.2 Training and Testing Strategy
- 4.2.3 Performance Evaluation Metrics

Objective 3: Model Performance Comparison

- 4.3.1 Confusion Matrix Analysis
- 4.3.2 ROC Curve and AUC Analysis
- 4.3.3 Accuracy Comparison

5. Conclusion

6. Future Scope

7. Linked IN Post and Google Drive Link

8. **References**

1. INTRODUCTION

1.1 Background of Predictive Analytics

Predictive analytics is a branch of data analytics that focuses on forecasting future outcomes by analyzing historical data using statistical methods and machine learning algorithms. Unlike descriptive analytics, which explains past events, predictive analytics aims to identify patterns and relationships that can be used to make informed predictions. In recent years, predictive analytics has gained significant importance in healthcare, where data-driven models assist in disease prediction, patient risk assessment, and clinical decision support.

Machine learning techniques such as classification algorithms enable the analysis of complex medical datasets and help uncover hidden patterns that may not be apparent through traditional statistical analysis. This project leverages predictive analytics techniques to analyze clinical data and predict mortality outcomes in heart failure patients.

1.2 Importance of Heart Failure Mortality Prediction

Heart failure is a severe cardiovascular condition that affects millions of individuals worldwide and is associated with high mortality rates. Early identification of patients at high risk of death can help healthcare professionals take timely preventive and therapeutic measures. Accurate mortality prediction can support clinical decision-making, improve patient monitoring, and optimize healthcare resource allocation.

By using machine learning models to analyze patient clinical records, mortality prediction becomes more systematic and data-driven. This approach enhances diagnostic accuracy and contributes to improved patient outcomes.

1.3 Motivation Behind the Project

The motivation behind this project is to apply predictive analytics concepts to a real-world healthcare dataset with high practical relevance. Heart failure data presents realistic challenges such as class imbalance, feature scaling, and model evaluation, making it an ideal case study for machine learning implementation.

This project also aims to gain hands-on experience in building, evaluating, and comparing multiple classification models using Python and scikit-learn. Through this work, theoretical knowledge of predictive analytics is transformed into practical understanding.

1.4 Problem Statement

Heart failure mortality prediction is challenging due to the complex interaction of multiple clinical parameters such as age, ejection fraction, serum creatinine, and comorbidities. Traditional analytical approaches may fail to capture these interactions effectively.

The problem addressed in this project is to develop a machine learning–based predictive framework capable of accurately classifying heart failure patients into survival and mortality outcomes using clinical records.

1.5 Objectives of the Study

The objectives of this study are:

- To explore and understand heart failure clinical data
 - To preprocess and prepare the dataset for machine learning
 - To implement multiple classification algorithms for mortality prediction
 - To evaluate and compare model performance using standard metrics
 - To identify the most effective predictive model
-

1.6 Scope of the Project

The scope of this project is limited to supervised machine learning classification using the Heart Failure Clinical Records dataset. The study focuses on mortality prediction based on available clinical attributes and does not include real-time patient monitoring or external medical data sources.

2. SOURCE OF DATASET

2.1 Overview of the Dataset

The dataset used in this project is the **Heart Failure Clinical Records Dataset**, which contains medical records of patients diagnosed with heart failure. It includes demographic information, clinical test results, and comorbid conditions.

2.2 Dataset Origin and Relevance

The dataset is publicly available and widely used for academic and research purposes

in predictive healthcare analytics. Its relevance lies in its real-world clinical attributes and clearly defined target variable for mortality prediction.

Link of Dataset:- <https://www.kaggle.com/datasets/andrewmvd/heart-failure-clinical-data/data>

2.3 Description of Data Attributes

The dataset includes attributes such as:

- Age
- Anaemia
- Diabetes
- Ejection fraction
- Serum creatinine
- Serum sodium
- High blood pressure
- Smoking
- Time
- DEATH_EVENT (target variable)

These features are clinically significant indicators of heart health.

	A	B	C	D	E	F	G	H	I	J	K	L	M	N
1	age	anaemia	creatinine	diabetes	ejection_fi	high_blood	platelets	serum_cre	serum_soc	sex	smoking	time	DEATH_EVENT	
2	75	0	582	0	20	1	265000	1.9	130	1	0	4	1	
3	55	0	7861	0	38	0	263358	1.1	136	1	0	6	1	
4	65	0	146	0	20	0	162000	1.3	129	1	1	7	1	
5	50	1	111	0	20	0	210000	1.9	137	1	0	7	1	
6	65	1	160	1	20	0	327000	2.7	116	0	0	8	1	
7	90	1	47	0	40	1	204000	2.1	132	1	1	8	1	
8	75	1	246	0	15	0	127000	1.2	137	1	0	10	1	
9	60	1	315	1	60	0	454000	1.1	131	1	1	10	1	
10	65	0	157	0	65	0	263358	1.5	138	0	0	10	1	
11	80	1	123	0	35	1	388000	9.4	133	1	1	10	1	
12	75	1	81	0	38	1	368000	4	131	1	1	10	1	
13	62	0	231	0	25	1	253000	0.9	140	1	1	10	1	
14	45	1	981	0	30	0	136000	1.1	137	1	0	11	1	
15	50	1	168	0	38	1	276000	1.1	137	1	0	11	1	
16	49	1	80	0	30	1	427000	1	138	0	0	12	0	
17	82	1	379	0	50	0	47000	1.3	136	1	0	13	1	
18	87	1	149	0	38	0	262000	0.9	140	1	0	14	1	
19	45	0	582	0	14	0	166000	0.8	127	1	0	14	1	
20	70	1	125	0	25	1	237000	1	140	0	0	15	1	
21	48	1	582	1	55	0	87000	1.9	121	0	0	15	1	
22	65	1	52	0	25	1	276000	1.3	137	0	0	16	0	
23	65	1	128	1	30	1	297000	1.6	136	0	0	20	1	
24	68	1	220	0	35	1	289000	0.9	140	1	1	20	1	
25	53	0	63	1	60	0	368000	0.8	135	1	0	22	0	
26	75	0	582	1	30	1	263358	1.83	134	0	0	23	1	
27	80	0	148	1	38	0	149000	1.9	144	1	1	23	1	
28	65	1	111	0	20	0	210000	1.9	137	1	0	7	1	

2.4 Data Size and Structure

The dataset consists of **299 records** and **13 independent variables** along with one binary target variable. The data is structured in tabular format, making it suitable for supervised machine learning techniques.

2.5 Assumptions and Limitations of the Dataset

The dataset assumes accurate and consistent medical data collection. Limitations include a relatively small dataset size and absence of real-time clinical updates. External factors affecting patient outcomes are not included.

3. DATASET PREPROCESSING

3.1 Need for Data Preprocessing

Data preprocessing is essential to ensure data quality and improve model performance. Medical datasets often contain variations in scale and distribution that must be addressed before modeling.

3.2 Handling Missing Values

The dataset was examined for missing values. Since the dataset contained no significant missing entries, no imputation was required, ensuring data consistency.

3.3 Data Cleaning and Validation

Data types were validated, and logical consistency checks were performed. The dataset was verified to ensure that all attributes were within realistic medical ranges.

File Edit Shell Debug Options Window Help

Python 3.13.9 (tags/v3.13.9:8183fa5, Oct 14 2025, 14:09:13) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>>

= RESTART: C:\Users\bably\OneDrive\Desktop\Predictive Project\Predictive Project.py

age anaemia creatinine_phosphokinase ... smoking time DEATH_EVENT

0 75.0 0 582 ... 0 4 1

1 55.0 0 7861 ... 0 6 1

2 65.0 0 146 ... 1 7 1

3 50.0 1 111 ... 0 7 1

4 65.0 1 160 ... 0 8 1

[5 rows x 13 columns]

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 299 entries, 0 to 298

Data columns (total 13 columns):

Column Non-Null Count Dtype

0 age 299 non-null float64

1 anaemia 299 non-null int64

2 creatinine_phosphokinase 299 non-null int64

3 diabetes 299 non-null int64

4 ejection_fraction 299 non-null int64

5 high_blood_pressure 299 non-null int64

6 platelets 299 non-null float64

7 serum_creatinine 299 non-null float64

8 serum_sodium 299 non-null int64

9 sex 299 non-null int64

10 smoking 299 non-null int64

11 time 299 non-null int64

12 DEATH_EVENT 299 non-null int64

dtypes: float64(3), int64(10)

memory usage: 30.5 KB

None

age anaemia ... time DEATH_EVENT

count 299.000000 299.000000 ... 299.000000 299.000000

mean 60.833893 0.431438 ... 130.260870 0.32107

std 11.894809 0.496107 ... 77.614208 0.46767

min 40.000000 0.000000 ... 4.000000 0.000000

25% 51.000000 0.000000 ... 73.000000 0.000000

50% 60.000000 0.000000 ... 115.000000 0.000000

75% 70.000000 1.000000 ... 203.000000 1.000000

max 95.000000 1.000000 ... 285.000000 1.000000

[8 rows x 13 columns]

3.4 Feature Scaling and Normalization

Feature scaling was applied using StandardScaler for models sensitive to feature magnitude, such as Logistic Regression, KNN, and SVM. Decision Tree and Naive

Bayes models were trained on unscaled data.

3.5 Final Dataset Preparation

The dataset was split into training and testing sets using an 80–20 ratio with stratification to maintain class balance. The prepared dataset was then used for model training and evaluation.

4. ANALYSIS ON DATASET

Objective 1: Exploratory Data Analysis of Heart Failure Data

4.1.1 General Description

Exploratory Data Analysis (EDA) was performed to understand feature distributions, relationships, and class imbalance in the dataset.

4.1.2 Analysis Results

EDA revealed that attributes such as age, ejection fraction, and serum creatinine have strong influence on mortality outcomes.

File Edit Format Run Options Window Help

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.preprocessing import StandardScaler

from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.tree import DecisionTreeClassifier
from sklearn.svm import SVC

from sklearn.metrics import (
    accuracy_score,
    confusion_matrix,
    classification_report,
    roc_curve,
    auc
)

sns.set(style="whitegrid")
df = pd.read_csv(
    "heart_failure_clinical_records_dataset.csv"
)

print(df.head())
print(df.info())
print(df.describe())
X = df.drop("DEATH_EVENT", axis=1)
y = df["DEATH_EVENT"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
models = {
```

Objective 2: Machine Learning Model Development

4.2.1 Models Implemented

The following models were implemented:

- Logistic Regression

```
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
models = {

    "Logistic Regression": LogisticRegression(
        penalty="l2",
        C=0.5,                      # regularization strength
        solver="liblinear",
        class_weight="balanced",
        max_iter=2000,
        random_state=42
    ),

    --- Logistic Regression ---
    Accuracy: 0.8

        precision    recall  f1-score   support

         0         0.82    0.90    0.86         41
         1         0.73    0.58    0.65         19

    accuracy          0.80         60
    macro avg         0.78    0.74    0.75         60
    weighted avg         0.79    0.80    0.79         60
```

- K-Nearest Neighbors

```
"KNN": KNeighborsClassifier(
    n_neighbors=11,                # smoother decision boundary
    weights="distance",           # closer points matter more
    metric="minkowski",
    p=2                           # Euclidean distance
),
```

```

--- KNN ---
Accuracy: 0.7333333333333333

```

	precision	recall	f1-score	support
0	0.74	0.95	0.83	41
1	0.71	0.26	0.38	19
accuracy			0.73	60
macro avg	0.73	0.61	0.61	60
weighted avg	0.73	0.73	0.69	60

- Naive Bayes

```

"Naive Bayes": GaussianNB(
    var_smoothing=1e-8      # improves numerical stability
),

```

```

--- Naive Bayes ---
Accuracy: 0.7333333333333333

```

	precision	recall	f1-score	support
0	0.75	0.93	0.83	41
1	0.67	0.32	0.43	19
accuracy			0.73	60
macro avg	0.71	0.62	0.63	60
weighted avg	0.72	0.73	0.70	60

- Decision Tree

```

"Decision Tree": DecisionTreeClassifier(
    criterion="entropy",
    max_depth=6,
    min_samples_split=10,
    min_samples_leaf=5,
    class_weight="balanced",
    random_state=42
),

```



```

--- Decision Tree ---
Accuracy: 0.7666666666666667
      precision    recall  f1-score   support

         0         0.85        0.80        0.82         41
         1         0.62        0.68        0.65         19

   accuracy
macro avg         0.73        0.74        0.74         60
weighted avg         0.77        0.77        0.77         60

```

- Support Vector Machine

```

"SVM": SVC(
    kernel="rbf",          # nonlinear decision boundary
    C=1.5,
    gamma="scale",
    probability=True,
    class_weight="balanced",
    random_state=42
)

--- SVM ---
Accuracy: 0.7166666666666667
      precision    recall  f1-score   support

         0         0.79        0.80        0.80         41
         1         0.56        0.53        0.54         19

   accuracy
macro avg         0.67        0.67        0.67         60
weighted avg         0.71        0.72        0.71         60

```

4.2.2 Training and Testing Strategy

Models were trained on the training dataset and evaluated on unseen test data. Stratified sampling ensured balanced class distribution.

4.2.3 Performance Evaluation Metrics

Model performance was evaluated using:

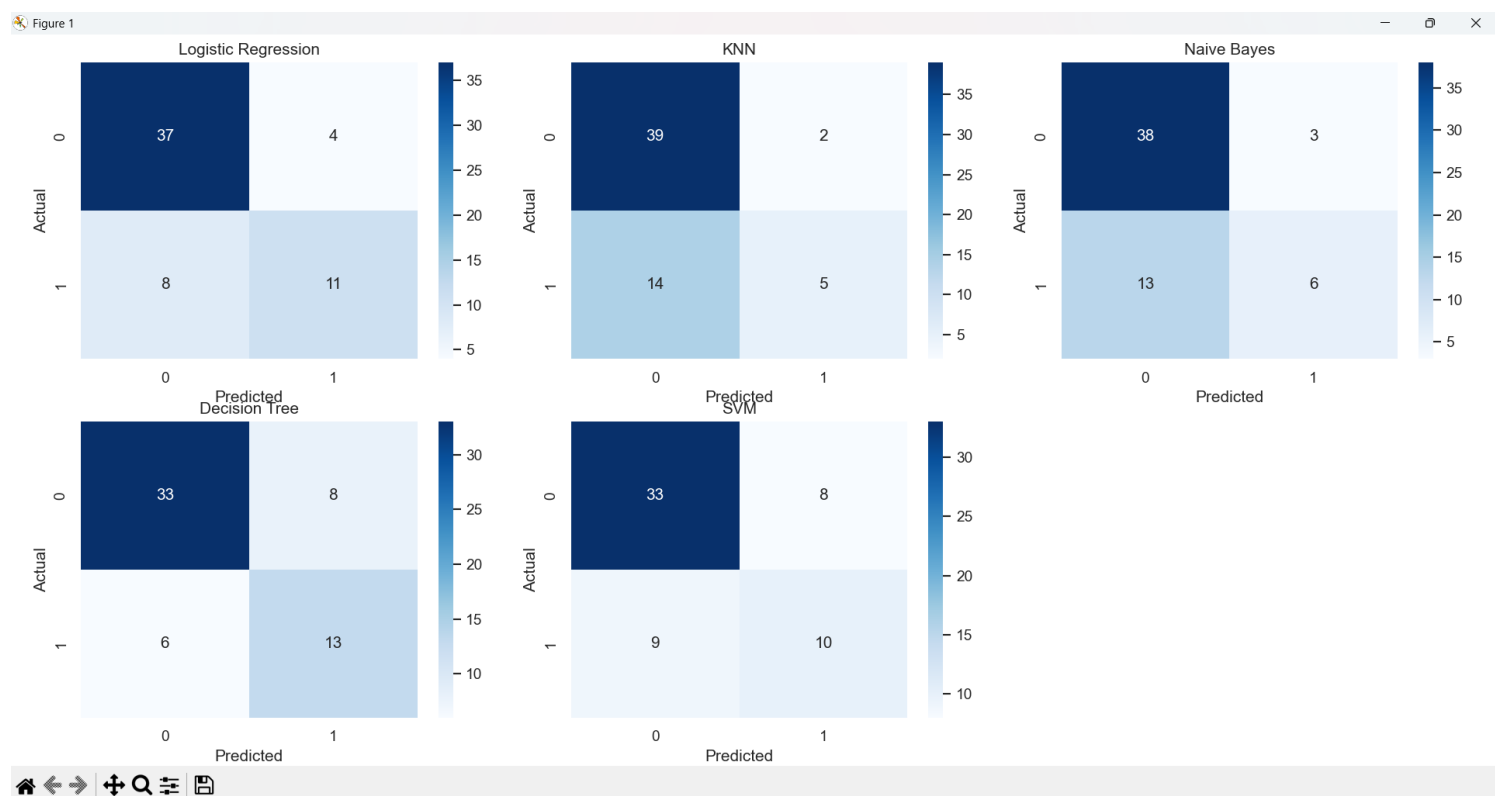
- Accuracy

- Precision, Recall, and F1-score
- Confusion Matrix
- ROC Curve and AUC

Objective 3: Model Performance Comparison

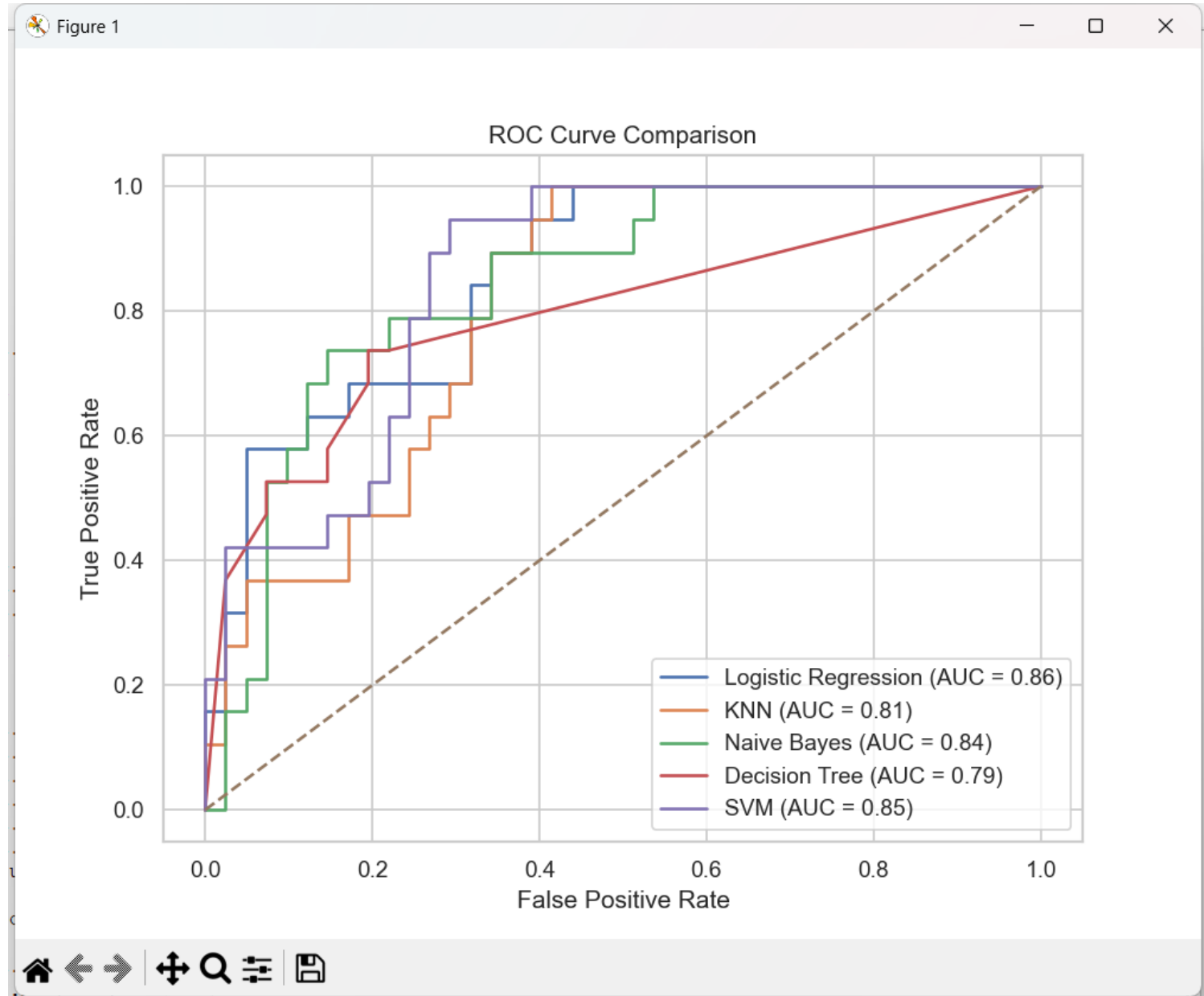
4.3.1 Confusion Matrix Analysis

Confusion matrices were analyzed to evaluate classification errors, including false positives and false negatives.



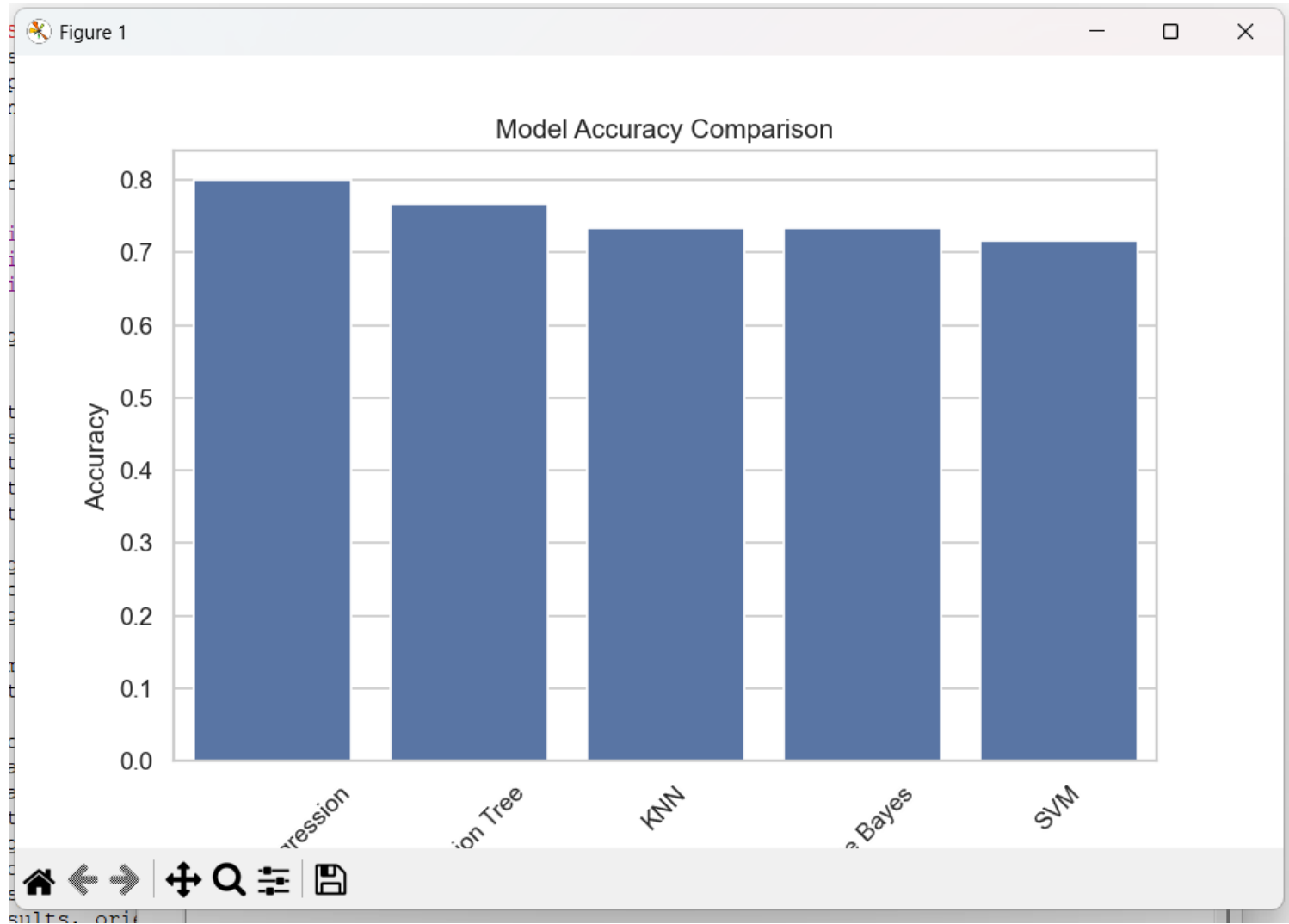
4.3.2 ROC Curve and AUC Analysis

ROC curves were plotted for all models to evaluate discrimination ability. AUC values were used to compare overall performance.



4.3.3 Accuracy Comparison

Model accuracies were compared using a bar chart to identify the best-performing classifier.



5. CONCLUSION

This project demonstrates the effective use of machine learning models in predicting heart failure mortality. The results confirm that predictive analytics can provide valuable insights for healthcare decision-making when combined with proper preprocessing and evaluation.

6. FUTURE SCOPE

Future enhancements may include:

- Use of ensemble models such as Random Forest
- Hyperparameter tuning using GridSearchCV
- Integration with real-time clinical systems
- Deployment using web-based applications

7. LinkedIn Post and Link



Saurabh Kumar • You

Engineering with Intent | Data Science Student | Applied ML | Data Engineering
1d • 🌐

I'm excited to share my recent project on Heart Failure Prediction using Machine Learning in Python.

In this project, I analyzed clinical data to predict patient mortality using multiple algorithms including Logistic Regression, KNN, Naive Bayes, Decision Tree, and SVM. I performed data preprocessing, scaling, and train-test splitting, and evaluated models using accuracy, confusion matrices, and ROC-AUC curves.

Key highlights:

Comparative analysis of multiple machine learning models

Visualization of model performance with heatmaps, ROC curves, and accuracy charts

Deriving actionable insights for healthcare prediction

This project enhanced my skills in Python, machine learning, data preprocessing, and model evaluation, while applying analytics to real-world healthcare data.

[#Python](#) [#MachineLearning](#) [#HealthcareAnalytics](#) [#DataScience](#) [#Projects](#)
[#Analytics](#) [#LearningByDoing](#) [#PredictiveAnalytics](#)

Predictive Project.py - C:\Users\babyl\OneDrive\Desktop\Predictive Project\Predictive Project.py (3.13.9)

File Edit Format Run Options Window Help

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.preprocessing import StandardScaler

from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.tree import DecisionTreeClassifier
from sklearn.svm import SVC

from sklearn.metrics import (
    accuracy_score,
    confusion_matrix,
    classification_report,
    roc_curve,
```

```
def plot_decision_boundary(model, X, y):
    """Plot the decision boundary for a 2D classification model.
    Parameters:
    model: A trained classification model.
    X: Feature matrix (array-like).
    y: Target labels (array-like).
    """
    # Create a mesh of points to plot
    x_min, x_max = X[:, 0].min() - 1, X[:, 0].max() + 1
    y_min, y_max = X[:, 1].min() - 1, X[:, 1].max() + 1
    h = 0.02
    xx, yy = np.meshgrid(np.arange(x_min, x_max, h),
                          np.arange(y_min, y_max, h))
    # Predict the class for each point in the mesh
    Z = model.predict(np.c_[xx.ravel(), yy.ravel()])
    # Put the result into a color map
    Z = Z.reshape(xx.shape)
    plt.figure(figsize=(10, 10))
    plt.pcolormesh(xx, yy, Z)
    # Scatter the original data points
    plt.scatter(X[:, 0], X[:, 1], c=y, s=50, edgecolor='k')
    plt.title('Decision Boundary')
    plt.xlabel('X1')
    plt.ylabel('X2')
    plt.show()

# Example usage:
# model = LogisticRegression()
# model.fit(X_train, y_train)
# plot_decision_boundary(model, X_test, y_test)
```

